Detecting Credit Card Fraud with Machine Learning: Per-Model Tuning of Sampling Strategies for Class Imbalance

Syed Zaheed
SRN: PES2UG24AM168
PES University, Bengaluru, India

Laxmikant
SRN: PES2UG24AM206
PES University, Bengaluru, India

***Abstract*—Credit card fraud causes substantial financial losses, yet fraudulent transactions are extremely rare, which makes it hard for standard classifiers to learn what fraud looks like. This paper reproduces and extends an earlier CS229 study by Rosenbaum. Four classifiers (logistic regression, logistic regression with quadratic terms and L1 regularisation, a random forest, and a one-hidden-layer neural network) are compared under five sampling strategies using a two-stage tournament. Unlike the original study, which chose the sampling proportion using logistic regression alone, we tune it separately for every model. On the Kaggle Credit Card Fraud Detection dataset (284,807 transactions, 0.173% fraud), configurations are selected by validation AUPRC and evaluated once on a held-out test set containing 82 frauds. The random forest with SMOTE at a minority share of 0.1 achieved the best test AUPRC (0.8623) and F1 (0.8820), while quadratic LASSO without resampling reached an AUPRC of 0.8395 and the highest AUROC (0.9831). Logistic regression and the neural network, both with SMOTE at p=0.5, detected 87.8% of frauds but raised 430 and 554 false alarms respectively.**

***Index Terms*—credit card fraud detection, class imbalance, SMOTE, random undersampling, random oversampling, precision-recall curve, random forest, LASSO.**

I. INTRODUCTION

Credit card fraud causes substantial financial losses, yet fraudulent transactions make up only a tiny fraction of all transactions. This severe class imbalance makes it difficult for standard classifiers to learn what fraud looks like, because a model that labels every transaction as legitimate is already almost always correct.

In this project, we reproduce and extend "Detecting Credit Card Fraud with Machine Learning" by Rosenbaum [1]. We compare four classifiers under several sampling strategies for handling class imbalance. We also address a limitation of the original study: the paper selected the sampling proportion using only simple logistic regression, whereas we tune it separately for each model, since the best proportion may differ across model families.

II. DATASET

We use the Kaggle Credit Card Fraud Detection dataset [2], which contains credit card transactions made by European cardholders over two days. It has 284,807 transactions and 31 columns: `Time` (seconds since the first transaction), `Amount`, `Class` (1 = fraud, 0 = legitimate), and `V1` to `V28`, which are anonymised features produced by PCA to protect privacy. Only 492 transactions (0.173%) are fraud, so the data are highly imbalanced.

We split the data into 2/3 training, 1/6 validation and 1/6 test sets, stratified by class so that each set keeps the same fraud rate. The test set contains 47,468 transactions, of which 82 are fraud, and it is used only once, at the end.

III. METHODOLOGY

A. Tournament Procedure

Following the paper [1], we use a two-stage tournament procedure. In stage 1, each of four model types is trained on the training set under five sampling strategies. The model types are logistic regression, logistic regression with quadratic terms and L1 (LASSO) regularisation, a random forest, and a neural network with one hidden layer. The sampling strategies are no resampling, random undersampling, random oversampling, a hybrid of both, and SMOTE [3]. SMOTE is our Python substitute for ROSE [4], which is only available in R. For every resampling strategy, we vary the minority-class share p from 1% to 50%.

Configurations are compared on the validation set using the area under the precision-recall curve (AUPRC), because accuracy is misleading when 99.8% of transactions are legitimate and the precision-recall plot is more informative than the ROC plot on imbalanced data. In stage 2, the best configuration (strategy and p) for each model is retrained on the combined training and validation data and evaluated once on the test set.

B. Implementation

The project is written in Python using pandas, scikit-learn [5] and imbalanced-learn [6], and is hosted in a private GitHub repository. `data_utils.py` loads the data and creates the fixed, stratified train/validation/test split (random seed 42), and `sampling.py` implements the five sampling strategies for any minority share p. `experiment.py` is a shared runner that, for a given model, tries every sampling strategy and p value, trains the model, and records validation AUPRC to a CSV file in the results folder. Each model has its own small script (`run_logistic.py`, `run_quadratic_lasso.py`, `run_rf.py` and `run_nn.py`).

As in the paper, the random forest uses 50 trees and the neural network has one hidden layer of 3 units with logistic activation. The quadratic model expands the 30 features into 495 polynomial terms (30 linear, 30 squared and 435 pairwise interaction terms) and applies an L1 penalty. Features are standardised for the linear models and min-max scaled for the neural network.

IV. RESULTS AND DISCUSSION

Table I lists the best stage-1 setup chosen for each model together with its test AUROC and AUPRC after retraining on the training and validation data. Table II reports threshold-based metrics and Table III the confusion counts on the 47,386 legitimate and 82 fraudulent test transactions.

TABLE I
SELECTED SETUP AND TEST RANKING METRICS

Model	Setup	AUROC	AUPRC
Logistic	SMOTE, p=0.5	0.9689	0.7082
Quadratic+LASSO	None	0.9831	0.8395
Random Forest	SMOTE, p=0.1	0.9595	0.8623
Neural Net	SMOTE, p=0.5	0.9813	0.6830

TABLE II
TEST CLASSIFICATION METRICS

Model	Acc.	Sens.	Spec.	Prec.	F1
Logistic	0.9907	0.8780	0.9909	0.1434	0.2466
Quadratic+LASSO	0.9995	0.7927	0.9999	0.9028	0.8442
Random Forest	0.9996	0.8659	0.9998	0.8987	0.8820
Neural Net	0.9881	0.8780	0.9883	0.1148	0.2034

TABLE III
TEST CONFUSION COUNTS

Model	TN	FP	FN	TP
Logistic	46,956	430	10	72
Quadratic+LASSO	47,379	7	17	65
Random Forest	47,378	8	11	71
Neural Net	46,832	554	10	72

A. Overall Ranking

Ranked by test AUPRC, the random forest is first (0.8623), followed by quadratic LASSO (0.8395), logistic regression (0.7082) and the neural network (0.6830). The random forest also has the best F1 (0.8820) and the quadratic LASSO model is close behind (0.8442). Both catch most frauds (71 and 65 of 82) with very few false alarms (8 and 7), which gives precisions of about 0.90.

B. AUROC Versus AUPRC

The two ranking metrics disagree. The random forest has the lowest AUROC (0.9595) but the highest AUPRC, while the neural network has the second-highest AUROC (0.9813) but the lowest AUPRC. Because legitimate transactions outnumber fraud by almost 600 to 1, even a small false-positive rate produces many false alarms without moving AUROC much. This supports the choice of AUPRC for model selection.

Fig. 1 shows the corresponding curves. In the ROC plot, the four models are almost indistinguishable at low false-positive rates, and the random forest falls behind only at higher false-positive rates, which is where its lower AUROC comes from. In the precision-recall plot the picture is different: the random forest keeps precision close to 1.0 until a recall of roughly 0.55 and stays above the other models over most of the range, and quadratic LASSO is the next best. The neural network curve is jagged and dips to about 0.7 precision at low recall, meaning that some of its highest-scored transactions are false alarms, and the logistic curve is a near-straight line over its first part, indicating few distinct score levels at the top of its ranking.

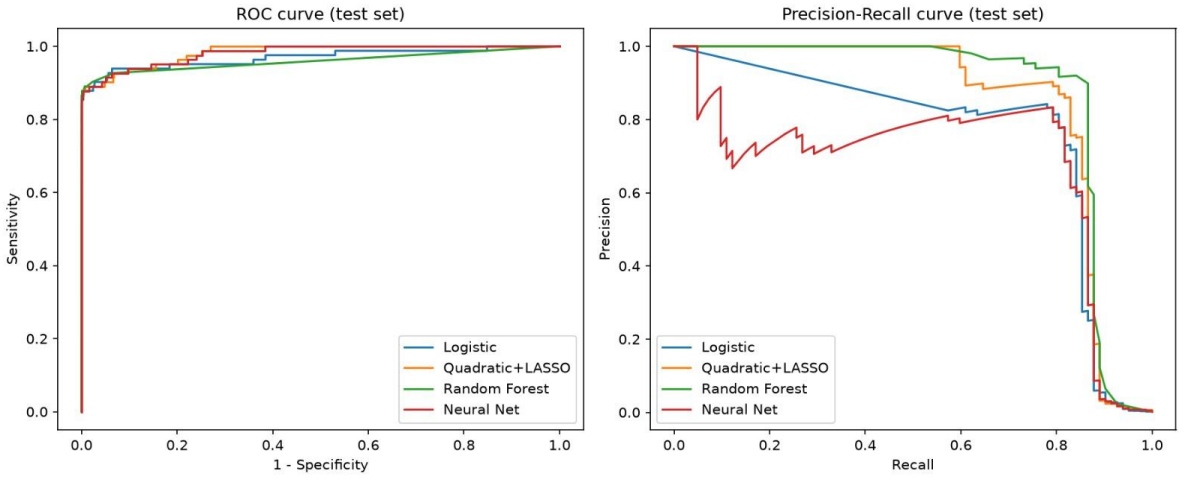


Fig. 1. ROC curve (left) and precision-recall curve (right) for the four models on the test set.

C. Effect of Resampling

The selected setups differ across models: SMOTE with $p = 0.1$ for the random forest, no resampling for quadratic LASSO, and SMOTE with $p = 0.5$ for logistic regression and the neural network. This is consistent with the motivation for tuning the proportion per model rather than fixing it from logistic regression alone. We note, however, that this study does not directly compare against a single shared proportion, so the size of the benefit is not measured here.

Logistic regression and the neural network reach the highest sensitivity (0.8780, 72 of 82 frauds) but at a large cost in precision: 430 and 554 legitimate transactions are flagged, giving precisions of only 0.143 and 0.115. Their accuracies (0.9907 and 0.9881) are even below the 0.9983 obtained by labelling every transaction legitimate, which illustrates why accuracy is a poor metric here. A likely explanation is that training on a 50% fraud mix inflates predicted fraud probabilities, so a default decision threshold flags too many transactions; recalibrating the threshold would be a natural follow-up, but we did not test it.

D. Limitations

The test set contains only 82 frauds, so differences of a few detections are within the noise: the random forest and quadratic LASSO differ by six detected frauds and one false alarm. Results come from a single split (seed 42). SMOTE is not equivalent to ROSE, so results may differ from the original R-based study. Finally, the data cover only two days and use PCA-anonymised features, which limits how well the findings transfer to other settings.

V. CONCLUSION

We reproduced and extended a tournament-style study of credit card fraud detection by tuning the sampling proportion separately for four model types. On the held-out test set, the random forest with SMOTE at $p = 0.1$ performed best on AUPRC (0.8623) and F1 (0.8820), and quadratic LASSO without resampling was a close second with higher precision and the best AUROC (0.9831). Logistic regression and the neural network with SMOTE at $p = 0.5$ detected more frauds but produced far more false alarms. The best resampling choice varied by model, supporting per-model tuning. Future work includes multiple random splits and confidence intervals, decision-threshold tuning, and a direct comparison with a single shared proportion.

REFERENCES

[1] Rosenbaum, "Detecting credit card fraud with machine learning," CS229 course project, Stanford Univ., Stanford, CA, USA, 2019.

[2] Machine Learning Group – ULB, "Credit card fraud detection," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[4] N. Lunardon, G. Menardi, and N. Torelli, "ROSE: A package for binary imbalanced learning," *R J.*, vol. 6, no. 1, pp. 79–89, 2014.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[6] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, 2017.